



Lecture 6

Normalisation

CSC1022 Introduction to Databases

CSC1005 Data Science & Databases
2025/2026 Semester 2

Prof. Mark Roantree
Uttaran Bera
School of Computing
@ Dublin City University

Agenda

- 1 Normalisation
- 2 First Normal Form: 1NF
- 3 Second Normal Form 2NF
- 4 Third Normal Form (3NF)



- What is the normalisation process?
- How are Functional Dependencies part of this process?
- How does one identify 1NF, 2F, 3NF?





- The normalisation process takes a relation schema through a series of tests to certify whether it satisfies a certain **normal form**.
- The process proceeds in a *top-down fashion* by evaluating each relation against the *criteria for normal forms* and **decomposing relations** as necessary
- It can be considered as *relational design by analysis*.
- There were initially 3 Normal Forms, all based on a single analytical tool: the **functional dependencies** among the attributes of a relation.



- Normalisation of data can be considered a process of analysing the given relation schemas based on their FDs and primary keys to achieve the desirable properties of
 - (1) minimising redundancy and
 - (2) minimising the insertion, deletion, and update anomalies.
- It can be considered as a *filtering* or *purification* process to ensure the design has successively better quality.
- Unsatisfactory relation schemas that do not meet certain conditions (the normal form tests) are *decomposed into smaller relation schemas* that meet the tests and thus, possess the desirable properties.



As a result, the normalisation procedure provides database designers with the following:

- A formal framework for analysing relation schemas based on their *keys* and on the *functional dependencies among their attributes*.
- A series of normal form tests that can be carried out on individual relation schemas so that the relational database can be normalised to any desired **degree**.



Definition. The **Normal Form** of a relation refers to the highest normal form condition that it meets and thus, indicates the **degree** to which it has been normalised.

- Normal forms, when considered in isolation from other factors, *do not guarantee* a good database design.
- It is generally not sufficient to check separately that each relation schema in the database is in 3NF.
- The process of normalisation through **decomposition** must *also* confirm the existence of additional properties that the relational schemas, taken together, should possess.



- The **nonadditive join** or lossless join property.
This guarantees that the spurious tuple generation problem does not occur with respect to the relation schemas created after decomposition.
- The **dependency preservation** property.
This ensures that each functional dependency is represented in some *individual* relation resulting after decomposition.

The *nonadditive join property* is critical and must be achieved at any cost, whereas the *dependency preservation property* is desirable (is sometimes sacrificed).



A **superkey** of a relation schema $R = A_1, A_2, \dots, A_n$ is a set of attributes $S \subseteq R$ with the property that no two tuples t_1 and t_2 in any legal relation state r of R will have $t_1[S] = t_2[S]$.

A **key** K is a superkey with the additional property that removal of any attribute from K will cause K not to be a superkey any more.

The difference between a key and a superkey is that a key has to be minimal: if we have a key $K = A_1, A_2, \dots, A_k$ of R , then $K - \{A_i\}$ is not a key of R for any A_i , $1 \leq i \leq k$.



An attribute of relation schema R is called a **prime attribute** of R if it is a member of some candidate key of R .

An attribute is called **nonprime** if it is not a *prime* attribute: if it is not a member of any candidate key.

First Normal Form (1NF)

First Normal Form (1NF)

[Normalisation](#)



[Normalisation](#)

[First Normal Form:
1NF](#)

[Second Normal Form
2NF](#)

[Third Normal Form
\(3NF\)](#)



Unnormalised Form (UNF)

A table that contains one or more repeating groups.

- *DreamHome* leases are shown in Figure 1.
- The lease on top is for client *John Kay*, leasing a property in *Glasgow*, owned by *Tina Murphy*.
- Here, we assume that a client rents a property only once and cannot rent more than one property at a time.
- Sample data from two leases for two different clients (John Kay and Aline Stewart) is transformed into table format (rows and columns) as per Figure 1.
- This is an example of an *unnormalised* table.

[Normalisation](#)[First Normal Form:
1NF](#)[Second Normal Form
2NF](#)[Third Normal Form
\(3NF\)](#)

Collecting Raw Data

<i>DreamHome Lease</i>	
<i>DreamHome Lease</i>	
<i>DreamHome Lease</i>	
<i>DreamHome Lease</i>	
Client Number <u>CR76</u> (Enter if known) Full Name <u>John Kay</u> (Please print)	Property Number <u>PG4</u> Property Address <u>6 Lawrence St, Glasgow</u>
Monthly Rent <u>350</u> Rent Start <u>01/07/03</u> Rent Finish <u>31/08/04</u>	Owner Number <u>C040</u> (Enter if known) Full Name <u>Tina Murphy</u> (Please print)

Figure 1: Collection of DreamHome leases

ClientRental Table

[Normalisation](#)



Collecting DreamHome into a table.

ClientRental

clientNo	cName	propertyNo	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	John Kay	PG4	6 Lawrence St, Glasgow	1-Jul-03	31-Aug-04	350	CO40	Tina Murphy
		PG16	5 Novar Dr, Glasgow	1-Sep-04	1-Sep-05	450	CO93	Tony Shaw
CR56	Aline Stewart	PG4	6 Lawrence St, Glasgow	1-Sep-02	10-June-03	350	CO40	Tina Murphy
		PG36	2 Manor Rd, Glasgow	10-Oct-03	1-Dec-04	375	CO93	Tony Shaw
		PG16	5 Novar Dr, Glasgow	1-Nov-05	10-Aug-06	450	CO93	Tony Shaw

[Normalisation](#)

[First Normal Form: 1NF](#)

[Second Normal Form 2NF](#)

[Third Normal Form \(3NF\)](#)

Figure 2: ClientRental Unnormalised Table

Transforming to 1NF

- 1 Identify the key attribute for the *ClientRental* unnormalised table as *clientNo*.
- 2 Identify the repeating group in the unnormalised table as the property rental details (repeats for each client). The structure of the repeating group is shown below.
- 3 Examine the multiple values at the intersection of certain rows and columns: two values for propertyNo (PG4 and PG16) for the client named John Kay.
- 4 Transform to 1NF: ensure a single value exists at each row/column intersection.

This is achieved by removing the repeating group.

ClientRental

clientNo	cName	propertyNo	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	John Kay	PG4	6 Lawrence St, Glasgow	1-Jul-03	31-Aug-04	350	CO40	Tina Murphy
		PG16	5 Novar Dr, Glasgow	1-Sep-04	1-Sep-05	450	CO93	Tony Shaw
CR56	Aline Stewart	PG4	6 Lawrence St, Glasgow	1-Sep-02	10-June-03	350	CO40	Tina Murphy
		PG36	2 Manor Rd, Glasgow	10-Oct-03	1-Dec-04	375	CO93	Tony Shaw
		PG16	5 Novar Dr, Glasgow	1-Nov-05	10-Aug-06	450	CO93	Tony Shaw

Figure 3: Repeating Groups

First Normal Form (1NF)

First Normal Form

A relation in which the intersection of each row and column contains one and only one value.



ClientRental

clientNo	propertyNo	cName	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	PG4	John Kay	6 Lawrence St, Glasgow	1-Jul-03	31-Aug-04	350	CO40	Tina Murphy
CR76	PG16	John Kay	5 Novar Dr, Glasgow	1-Sep-04	1-Sep-05	450	CO93	Tony Shaw
CR56	PG4	Aline Stewart	6 Lawrence St, Glasgow	1-Sep-02	10-Jun-03	350	CO40	Tina Murphy
CR56	PG36	Aline Stewart	2 Manor Rd, Glasgow	10-Oct-03	1-Dec-04	375	CO93	Tony Shaw
CR56	PG16	Aline Stewart	5 Novar Dr, Glasgow	1-Nov-05	10-Aug-06	450	CO93	Tony Shaw

Figure 4: First Normal Form ClientRental Relation

- We removed the repeating group (property rented details) by entering the appropriate client data *into its own row*.
- The resulting first normal form *ClientRental* relation is shown in Figure 4.





Second Normal Form (2NF)

Functional Dependencies

ClientRental

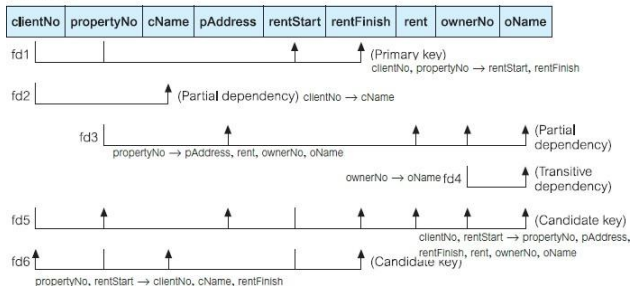


Figure 5: FDs of the ClientRental Relation

- Use FDs to identify candidate keys for the *ClientRental* relation. The result is composite keys: **(clientNo, propertyNo)**, **(clientNo, rentStart)**, and **(propertyNo, rentStart)**.
- We select **(clientNo, propertyNo)** as the primary key for the relation, and place the PK at the left-hand side of the relation.

- FD1 shows the primary key: neither *clientNo* nor *propertno* alone provide uniqueness and so (*clientNo*,*propertno*) is selected.
- FD2 highlights a **partial dependency** problem as *clientNo* (alone) determines *cName*.
- FD3 highlights a **partial dependency** problem as *propertno* (alone) determines a number of property attributes.
- FD4 highlights a **transitive dependency** problem through *propertno*.
- FDs 5 and 6 show candidate keys.



- The ClientRental relation is in 1NF as there is a single value at the intersection of each row and column.
- The relation contains data describing clients, property rented, and property owners. This is repeated several times: significant redundancy!
- If implemented, the 1NF relation would be subject to update anomalies.
- To remove some of these, we must transform the relation into Second Normal Form.



- Remove the repeating group (client name) together with a copy of the original key attribute (clientNo) into a separate relation, as shown in Figure 6.
- With the help of the functional dependencies supplied in Figure 5, identify the primary keys.
- The format of the resulting 1NF relations are as follows:
Client (*clientNo*, *cName*)
PropertyRentalOwner (*clientNo*, *propertyNo*, *pAddress*, *rentStart*, *rentFinish*, *rent*, *ownerNo*, *oName*)



ClientRental Normalised but are new tables 2NF?

[Normalisation](#)



Client

clientNo	cName
CR76	John Kay
CR56	Aline Stewart

[Normalisation](#)

[First Normal Form:
1NF](#)

[Second Normal Form
2NF](#)

[Third Normal Form
\(3NF\)](#)

PropertyRentalOwner

clientNo	propertyNo	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	PG4	6 Lawrence St, Glasgow	1-Jul-03	31-Aug-04	350	CO40	Tina Murphy
CR76	PG16	5 Novar Dr, Glasgow	1-Sep-04	1-Sep-05	450	CO93	Tony Shaw
CR56	PG4	6 Lawrence St, Glasgow	1-Sep-02	10-Jun-03	350	CO40	Tina Murphy
CR56	PG36	2 Manor Rd, Glasgow	10-Oct-03	1-Dec-04	375	CO93	Tony Shaw
CR56	PG16	5 Novar Dr, Glasgow	1-Nov-05	10-Aug-06	450	CO93	Tony Shaw

Figure 6: 1NF Client and PropertyRentalOwner relations.

- The *Client* and *PropertyRentalOwner* relations are both in 1NF as there is a single value at the intersection of each row and column.
- The *Client* relation contains data describing clients and the *PropertyRentalOwner* relation contains data describing property rented by clients and property owners.
- However, from Figure 6, we see this relation also contains some redundancy and as a result may suffer from similar update anomalies.





A relation that is in 1NF and every non-primary key attribute is *fully functionally dependent* on the primary key.

- Second Normal Form (2NF) is based on the concept of **full functional dependency**.
- Thus, 2NF applies to relations with composite keys: a primary key composed of two or more attributes.
- A relation with a single-attribute primary key must (at least) be 2NF!
- A relation that is not in 2NF may suffer from the update anomalies: if we wish to change the rent of property number PG4, we have to update two tuples in the *ClientRental* relation in Figure 6.
- If only one tuple is updated with the new rent, this results in an inconsistency in the database.

Partial Dependencies

- The normalisation of 1NF relations to 2NF involves the removal of **partial dependencies**.
- If a partial dependency exists, we remove the partially dependent attribute(s) from the relation by placing them in a new relation along with a copy of their *determinant*.
- We got rid of 1 problem but more partial dependencies remain.





PK is (clientNo,propertyNo) so potentially we have a problem!

fd1	clientNo, propertyNo $\rightarrow$ rentStart, rentFinish	(Primary key)
fd2	clientNo $\rightarrow$ cName	(Partial dependency)
fd3	propertyNo $\rightarrow$ pAddress, rent, ownerNo, oName	(Partial dependency)
fd4	ownerNo $\rightarrow$ oName	(Transitive dependency)
fd5	clientNo, rentStart $\rightarrow$ propertyNo, pAddress, rentFinish, rent, ownerNo, oName	(Candidate key)
fd6	propertyNo, rentStart $\rightarrow$ clientNo, cName, rentFinish	(Candidate key)

Figure 7: Functional Dependencies for ClientRental

[Normalisation](#)[First Normal Form:
1NF](#)[Second Normal Form
2NF](#)[Third Normal Form
\(3NF\)](#)



Using the set of FDs, normalise the *ClientRental* relation.

- 1 Test if the *ClientRental* relation is in 2NF, by identifying any *partial dependencies* on the primary key.
- 2 Note the client attribute (*cName*) is **partially dependent** on the primary key: on only the *clientNo* attribute (fd2).
- 3 The property attributes (*pAddress*, *rent*, *ownerNo*, *oName*) are **partially dependent** on the primary key: on only the *propertyNo* attribute (fd3).
- 4 The property rented attributes (*rentStart* and *rentFinish*) are **fully dependent** on the (entire) primary key: the *clientNo* and *propertyNo* attributes (fd1).



Transform *PropertyRental* to 2NF: create new relations so non-PK attributes are removed, along with a copy of *that part* of the PK on which they are *fully functionally dependent*.

- The identification of partial dependencies in *ClientRental* indicates that the relation is not in 2NF.
- The 2NF transformation creates 3 new relations called *Client*, *Rental*, and *PropertyOwner*, as shown in Figure 8.

2NF Relations

[Normalisation](#)

ClientRental becomes *Client*, *Rental*, and *PropertyOwner*



[Normalisation](#)

[First Normal Form:
1NF](#)

[Second Normal Form
2NF](#)

[Third Normal Form
\(3NF\)](#)

Client

clientNo	cName
CR76	John Kay
CR56	Aline Stewart

Rental

clientNo	propertyNo	rentStart	rentFinish
CR76	PG4	1-Jul-03	31-Aug-04
CR76	PG16	1-Sep-04	1-Sep-05
CR56	PG4	1-Sep-02	10-Jun-03
CR56	PG36	10-Oct-03	1-Dec-04
CR56	PG16	1-Nov-05	10-Aug-06

PropertyOwner

propertyNo	pAddress	rent	ownerNo	oName
PG4	6 Lawrence St, Glasgow	350	CO40	Tina Murphy
PG16	5 Novar Dr, Glasgow	450	CO93	Tony Shaw
PG36	2 Manor Rd, Glasgow	375	CO93	Tony Shaw

Figure 8: Second Normal Form relations derived from the *ClientRental* relation.

2NF Relations

The relations have the following form:

Client (clientNo, cName)

Rental (clientNo, propertyNo, rentStart, rentFinish)

PropertyOwner (propertyNo, pAddress, rent, ownerNo, oName)



Third Normal Form (3NF)

[Normalisation](#)



[Normalisation](#)

[First Normal Form:
1NF](#)

[Second Normal Form
2NF](#)

[Third Normal Form
\(3NF\)](#)

A background image of a field of tall, green and yellow grass, possibly a meadow or a field of wildflowers, with the text 'Third Normal Form (3NF)' overlaid in white.

Third Normal Form (3NF)

- Although 2NF relations have less redundancy, they may still suffer from update anomalies.
- To update the name of an owner, such as Tony Shaw (ownerNo CO93), we must update two tuples in the *PropertyOwner* relation of Figure 8.
- If one tuple were updated but not the second, the database would be in an inconsistent state.
- This update anomaly is caused by a **transitive dependency**.
- Remove these dependencies to progress to **Third Normal Form**.





A relation that is in First and Second Normal Form and in which no non-primary key attribute is transitively dependent on the primary key.

- The normalisation of 2NF relations to 3NF involves the removal of *transitive dependencies*.
- If a transitive dependency exists, remove the transitively dependent attribute(s) by placing the attribute(s) in a new relation along with a copy of the *determinant*.

- All non-PK attributes in *Client* and *Rental* are functionally dependent on *only* their primary keys.
- The *Client* and *Rental* relations have no transitive dependencies and are therefore already in 3NF.
- In figure 9, a functional dependency is labeled with a prime (such as fd5'), indicating that the dependency has altered from the original FD shown in Figure [5](#).
- All non-PK attributes in *PropertyOwner* are functionally dependent on the PK, with the exception of *oName*, which is transitively dependent on *ownerNo* (fd4).
- This transitive dependency was previously identified in Figure [5](#).



Normalisation



First Normal Form:
1NF

Second Normal Form

Third Normal Form (3NF)

(Primary key)

(Primary key)

(Candidate key)

(Candidate key)

(Primary key)

(Transitive dependency)

6.36

PropertyOwner is not in 3NF!

propertyNo → ownerNo
ownerNo → oName

PropertyOwner

propertyNo	pAddress	rent	ownerNo	oName
PG4	6 Lawrence St, Glasgow	350	CO40	Tina Murphy
PG16	5 Novar Dr, Glasgow	450	CO93	Tony Shaw
PG36	2 Manor Rd, Glasgow	375	CO93	Tony Shaw

Figure 10: Transitive Dependency.

- To transform the *PropertyOwner* relation into 3NF, first remove this transitive dependency by creating two new relations called *PropertyForRent* and *Owner* (figure 11).
- The new relations have the form:
PropertyForRent (propertyNo, pAddress, rent, ownerNo)
Owner (ownerNo, oName)
- The *PropertyForRent* and *Owner* relations are in 3NF as there are no further transitive dependencies on the primary key.



3NF Relations



PropertyForRent

propertyNo	pAddress	rent	ownerNo
PG4	6 Lawrence St, Glasgow	350	CO40
PG16	5 Novar Dr, Glasgow	450	CO93
PG36	2 Manor Rd, Glasgow	375	CO93

Owner

ownerNo	oName
CO40	Tina Murphy
CO93	Tony Shaw

Figure 11: 3NF relations derived from PropertyOwner.

ClientRental Decomposition

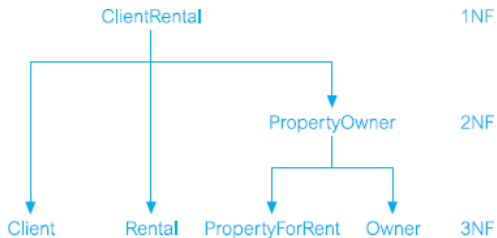


Figure 12: Decomposition of the 1NF ClientRental into 3NF relations.



- The *ClientRental* relation (Figure 3) has been transformed by the process of normalisation into *four* relations in 3NF.
- Figure 12 illustrates the process by which the original 1NF relation is decomposed into the 3NF relations.
- The resulting 3NF relations have the form:

Client (clientNo, cName)

Rental (clientNo, propertyNo, rentStart, rentFinish)

PropertyForRent (propertyNo, pAddress, rent, ownerNo)

Owner (ownerNo, oName)

Recreating the Unnormalised Relation

- The original *ClientRental* relation from Figure 3 can be recreated by joining the *Client*, *Rental*, *PropertyForRent*, and *Owner* relations (PK-FK joins).
- The *ownerNo* attribute is the PK for *Owner* and is FK in *PropertyForRent*.
- The *clientNo* attribute is the PK for *Client* and is FK in *Rental*.
- Here, the *clientNo* attribute in the *Rental* relation acts both as FK and as *part* of the primary key of this relation.



- Similarly, *propertyNo* is PK for *PropertyForRent* and is FK in *Rental* and part of the PK.
- One can see that the normalisation process decomposed the *ClientRental* relation using a series of algebra *project* commands.
- This results in a **lossless-join** decomposition, which is reversible using the *natural join* operation.



Client

clientNo	cName
CR76	John Kay
CR56	Aline Stewart

Rental

clientNo	propertyNo	rentStart	rentFinish
CR76	PG4	1-Jul-03	31-Aug-04
CR76	PG16	1-Sep-04	1-Sep-05
CR56	PG4	1-Sep-02	10-Jun-03
CR56	PG36	10-Oct-03	1-Dec-04
CR56	PG16	1-Nov-05	10-Aug-06

PropertyForRent

propertyNo	pAddress	rent	ownerNo
PG4	6 Lawrence St, Glasgow	350	CO40
PG16	5 Novar Dr, Glasgow	450	CO93
PG36	2 Manor Rd, Glasgow	375	CO93

Owner

ownerNo	oName
CO40	Tina Murphy
CO93	Tony Shaw

Figure 13: A summary of the 3NF relations derived from the ClientRental relation.

Lecture Ends ...



[Normalisation](#)



[Normalisation](#)

[First Normal Form:
1NF](#)

[Second Normal Form
2NF](#)

[Third Normal Form
\(3NF\)](#)